CLOUD & PLATFORMS

Microsoft Azure

Core services, architecture, IaaS/PaaS/SaaS, ARM/Bicep, DevOps, monitoring, and interview prep

azure cloud iaas paas devops compute interview-prep networking storage entra-id bicep arm aks monitoring

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. Azure Overview & Global Infrastructure
2. Subscriptions, Resource Groups & Management Hierarchy
3. Compute Services
4. Azure Storage Services
5. Azure Networking
6. Identity & Access Management (Entra ID / Azure AD)
7. IaaS vs PaaS vs SaaS
8. Azure Pricing Model & Cost Management
9. Azure CLI & Portal
10. Infrastructure as Code: ARM Templates & Bicep
11. Azure DevOps
12. Monitoring: Azure Monitor & Log Analytics
13. Security & Compliance Best Practices
14. Azure Cheat Sheet: Quick Reference
15. Interview Q&A

Azure Overview & Global Infrastructure

Microsoft Azure is the world's second-largest public cloud, operating in 60+ regions across 140+ countries. It offers over 200 products and services covering compute,

storage, networking, AI, databases, DevOps, and security — all accessible through a shared, pay-as-you-go infrastructure.

GEOGRAPHIES

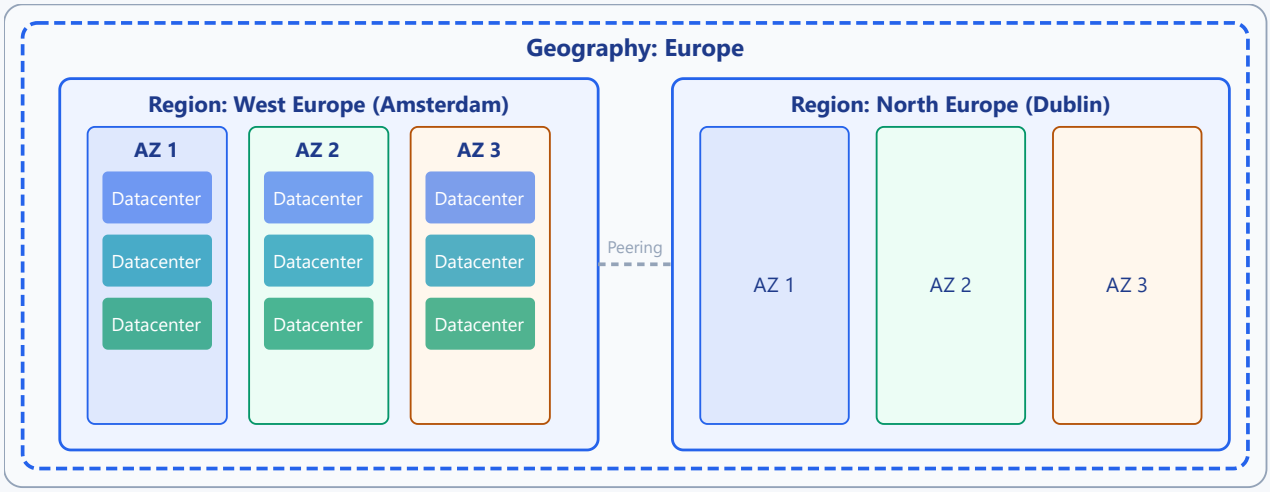
High-level boundaries (e.g., United States, Europe, Asia Pacific) that preserve data residency and compliance. Each geography contains 1+ regions.

REGIONS

Discrete data-center clusters within a geography. Examples: **East US** **West Europe** **Southeast Asia**. Each region is isolated from others to limit blast radius.

AVAILABILITY ZONES (AZS)

Physically separate data centers inside a region with independent power, cooling, and networking. At least 3 AZs per zone-enabled region. Latency between AZs is <2 ms.



Azure Geography → Region → Availability Zone hierarchy. AZs share low-latency interconnects but have independent power and cooling to eliminate single points of failure.

Region Pairs

Most Azure regions are paired with another region in the same geography at least 300 miles apart. During planned maintenance Microsoft updates only one region in a pair at a time. Region pairs are used for geo-redundant storage (GRS) replication and Azure Site Recovery.

Region	Paired With	Geography
East US	West US	United States
North Europe (Dublin)	West Europe (Amsterdam)	Europe
East Asia (Hong Kong)	Southeast Asia (Singapore)	Asia Pacific
Australia East	Australia Southeast	Australia
Brazil South	South Central US	Cross-geography

> **Note**
> Sovereign clouds (Azure Government, Azure China 21Vianet) are physically and logically isolated from the global Azure public cloud and have separate endpoints.

Subscriptions, Resource Groups & Management Hierarchy

Azure organizes resources in a four-level management hierarchy: Management Groups → Subscriptions → Resource Groups → Resources. Policies and RBAC roles applied at higher levels cascade down.



MANAGEMENT GROUPS

Container for multiple subscriptions. Used by enterprises to apply Azure Policy and RBAC at scale. Up to 6 levels of hierarchy, with the root "Tenant Root Group" at the top.

SUBSCRIPTIONS

Billing boundary and trust boundary. Each subscription is linked to an Entra ID tenant. Hard limits apply per subscription (e.g., 980 resource groups, 800 deployments per RG).

RESOURCE GROUPS

Logical container for related resources sharing the same lifecycle. Resources in a group can span multiple regions. Tags, policies, and RBAC apply at the group level.

RESOURCES

Individual services (VMs, storage accounts, databases, etc.). A resource belongs to exactly one resource group but can interact with resources in other groups or subscriptions.

Tip

Use separate resource groups per environment (dev/staging/prod) and per application. This makes cost tracking, lifecycle management, and least-privilege RBAC straightforward.

Pitfall

Deleting a resource group deletes all resources inside it with no undo. Always use resource locks (`CanNotDelete` or `ReadOnly`) on production resource groups to prevent accidental deletion.

Azure Tags

Tags are key-value metadata pairs attached to resources or resource groups. They don't affect functionality but are critical for cost reporting, automation, and compliance. A resource can have up to 50 tags.

BASH

```
az tag create --resource-id /subscriptions/<sub-id>/resourceGroups/myRG \
  --tags Environment=Production Owner=TeamA CostCenter=CC-1234
```

Compute Services

Azure offers a continuum of compute options from full IaaS control (Virtual Machines) to fully managed serverless (Functions). Choosing the right tier trades control for operational overhead.

Azure Virtual Machines (VMs)

VMs are the fundamental IaaS building block. You pick an OS image, VM size (CPU/RAM), managed disks, and networking. Azure provides the hypervisor (Hyper-V) but you manage the OS, patches, and runtime.

VM SERIES CHEAT SHEET

Series	Optimized For	Example Sizes	Use Case
B (Burstable)	Variable CPU load	B2s, B4ms	Dev/test, low-traffic web
D (General)	Balanced CPU/RAM	D4s v5, D16s v5	Web servers, small databases
E (Memory)	High RAM	E8s v5, E64s v5	SAP HANA, in-memory caches
F (Compute)	High CPU	F8s v2, F72s v2	Batch processing, gaming
N (GPU)	GPU	NC6s v3, ND96asr A100	ML training, rendering
L (Storage)	High disk throughput	L8s v3, L48s v3	NoSQL, data warehousing

AVAILABILITY OPTIONS

AVAILABILITY SETS

Groups VMs across **fault domains** (separate racks) and **update domains** (sequential reboot groups). Protects against rack failures but stays within a single AZ. SLA: 99.95%.

AVAILABILITY ZONES

Deploy VMs across distinct physical datacenters in the same region. Best in-region protection. SLA: 99.99%. Use zone-redundant storage for disks.

VM SCALE SETS (VMSS)

Manage a group of identical VMs with auto-scaling (metric-based or schedule). Integrates with Azure Load Balancer and Application Gateway. Supports Spot instances for cost savings.

✦ **Best Practice**

Attach managed disks (not unmanaged) to all VMs. Use Premium SSD for production workloads. Enable Azure Backup and Azure Site Recovery for DR. Use Spot VMs for fault-tolerant batch workloads at up to 90% savings.

Azure App Service

A fully managed PaaS platform for hosting web apps, REST APIs, and mobile backends. Supports .NET, Java, Node.js, Python, PHP, Ruby, and custom containers. No VM management required.

Plan Tier	Features	Scaling	Use Case
Free / Shared	Shared infra, no custom domain SSL	None	Dev/exploration
Basic (B1–B3)	Dedicated infra, custom domains, SSL	Manual (1–3 instances)	Dev/test
Standard (S1–S3)	Auto-scale, staging slots, daily backups	Auto (up to 10)	Production
Premium v3 (P1v3–P3v3)	VNet integration, no cold start, more RAM	Auto (up to 30)	Enterprise production
Isolated v2	Dedicated VNet, VNET injection, ASE	Auto (up to 100)	High-security, compliance

Deployment slots allow blue-green deployments — traffic can be gradually shifted from staging to production, then swapped atomically with swap keeping connection strings and app settings pinned to each slot.

BASH

```bash
# Deploy a Node.js app from GitHub Actions or local zip
az webapp deployment source config-zip \
  --resource-group myRG \
  --name myAppService \
  --src app.zip

# Swap staging slot to production
az webapp deployment slot swap \
  --resource-group myRG \
  --name myAppService \
  --slot staging \
  --target-slot production
```

Azure Functions

Event-driven, serverless compute that runs code in response to triggers (HTTP, Timer, Storage queues, Event Hubs, Cosmos DB change feed, etc.). Billed per execution (first 1 million free) and GB-seconds of memory consumption.

CONSUMPTION PLAN

True serverless — pay only when running. Auto-scales to zero. Cold start latency (typically 200–500 ms for .NET). Max execution timeout: 5 minutes default, 10 minutes max.

PREMIUM PLAN

Pre-warmed workers eliminate cold starts. VNet integration. Unlimited execution duration. Higher baseline cost but predictable latency. Ideal for mission-critical functions.

PYTHON

```python
import azure.functions as func
import logging

app = func.FunctionApp(http_auth_level=func.AuthLevel.FUNCTION)

@app.route(route="hello")
def http_trigger(req: func.HttpRequest) -> func.HttpResponse:
    name = req.params.get("name", "World")
    return func.HttpResponse(f"Hello, {name}!", status_code=200)
```

Azure Kubernetes Service (AKS)

Managed Kubernetes: Azure provisions and manages the control plane (API server, etcd, scheduler) at no charge. You pay only for the agent node VMs. AKS integrates deeply with Entra ID for RBAC, Azure CNI for networking, Azure Monitor for observability, and Azure Container Registry for images.

BASH

```
# Create an AKS cluster with 3 nodes in availability zones
az aks create \
  --resource-group myRG \
  --name myAKS \
  --node-count 3 \
  --zones 1 2 3 \
  --enable-managed-identity \
  --network-plugin azure \
  --enable-oidc-issuer \
  --enable-workload-identity \
  --generate-ssh-keys

# Get credentials
az aks get-credentials --resource-group myRG --name myAKS
```

> **Note**
> AKS node pools support multiple VM SKUs. Use system node pools for system pods and separate user node pools for workloads. Enable cluster autoscaler on user pools for cost-efficient auto-scaling.

Azure Storage Services

Every Azure storage account is an abstraction over distributed, durable object storage. All data is replicated at least 3 times. A single storage account supports Blob, Table, Queue, and Files simultaneously.

Storage Account Redundancy Options

Redundancy	Copies	Failure Scope	SLA (read/write)
LRS Locally Redundant	3 in 1 datacenter	Single device/rack	99.9% / 99.9%
ZRS Zone Redundant	3 across AZs	Single AZ	99.9% / 99.9%
GRS Geo-Redundant	6 (3+3 in pair)	Region failure	99.9% / 99.9%
GZRS Geo-Zone Redundant	6 (ZRS+3 in pair)	AZ + region failure	99.99% read / 99.9% write
RA-GRS Read-Access Geo	6 (GRS + secondary read)	Region failure	99.99% read / 99.9% write

Blob Storage

Object/blob storage for unstructured data (images, videos, backups, logs, static websites). Organized as: Storage Account → Container → Blob.

BLOCK BLOB

Most common type. Optimized for streaming and storing text or binary files. Supports upload from 100 MB chunks. Max blob size: 190.7 TB.

APPEND BLOB

Optimized for append operations only. Ideal for logging scenarios where data is written sequentially. Cannot modify existing blocks.

PAGE BLOB

Used internally for Azure VM managed disks. Supports random read/write access in 512-byte pages. Max size: 8 TB.

ACCESS TIERS

Tier	Storage Cost	Access Cost	Best For
Hot	High	Low	Frequently accessed data
Cool	Medium	Medium	Infrequently accessed, stored 30+ days
Cold	Low	Higher	Rarely accessed, stored 90+ days
Archive	Very low	Very high + rehydrate time	Long-term backup, compliance (180+ days)

> ✦ **Tip**
> Use Lifecycle Management policies to automatically tier blobs to Cool/Archive after a defined number of days since modification. This can cut storage costs by 70%+ for log archives.

Azure Table Storage

A NoSQL key-value store for large amounts of structured, non-relational data. Each entity (row) has a PartitionKey and RowKey as the composite primary key. Queries on non-key attributes require full-partition scans — design your PartitionKey carefully. For richer NoSQL, prefer Azure Cosmos DB.

Azure Queue Storage

Simple message queue for decoupling application components. Messages up to 64 KB. Visibility timeout allows workers to claim a message without deleting it — if processing fails and timeout expires, the message becomes visible again. Max TTL: 7 days (configurable to -1 for infinite).

Azure Files

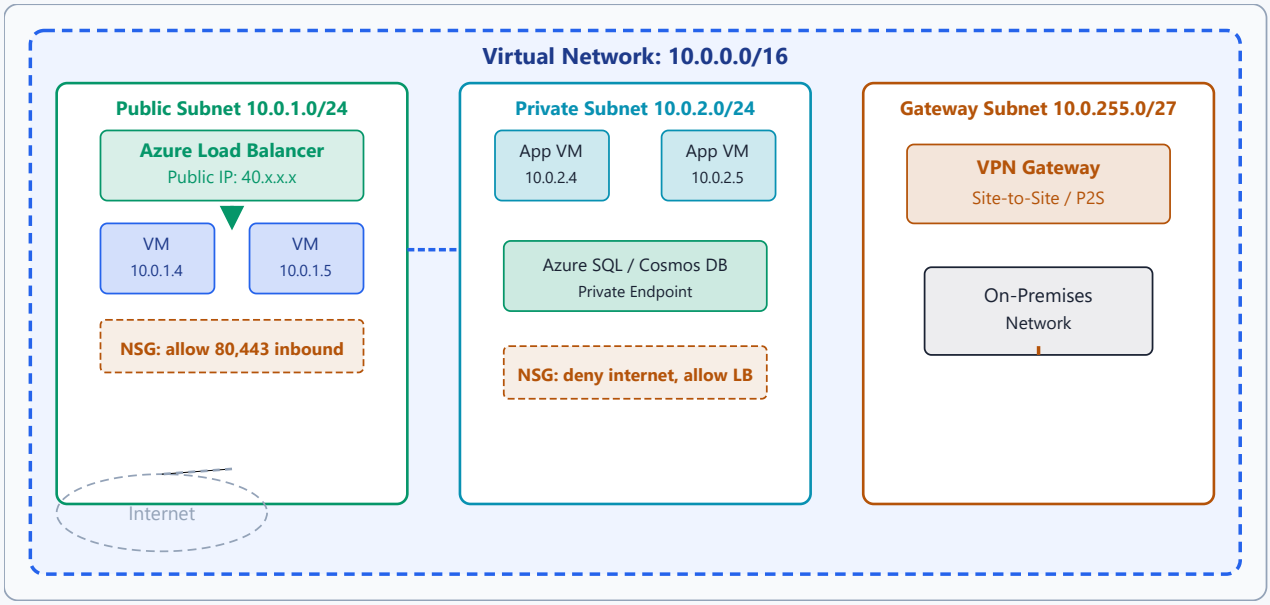
Fully managed file shares accessible over SMB 3.x and NFS 4.1. Can be mounted on Windows, Linux, and macOS. Azure File Sync allows caching shares on-premises Windows servers, enabling a cloud-tiered hybrid storage model.

BASH

```bash
# Mount an Azure file share on Linux
sudo mount -t cifs //<account>.file.core.windows.net/<share> /mnt/azfiles \
  -o vers=3.0,username=<account>,password=<key>,dir_mode=0777,file_mode=0777
```

Azure Networking

Azure Virtual Networks provide private, isolated network segments in the cloud. All connectivity — whether VNet-to-VNet, on-premises, or internet-facing — flows through well-defined networking constructs.



Three-tier Azure VNet architecture: public subnet (Load Balancer + web VMs), private subnet (app VMs + database via Private Endpoint), and gateway subnet for on-premises connectivity.

Key Networking Components

VIRTUAL NETWORK (VNET)

A logically isolated network with a defined private IP address space (CIDR). VNets are regional — they cannot span regions. Subnets segment the address space; each subnet must be within the VNet's CIDR.

NETWORK SECURITY GROUPS (NSGS)

Stateful Layer-4 firewall rules applied to subnets or individual NICs. Rules have priority (100–4096, lower = higher priority), direction (Inbound/Outbound), source/destination (IP, tag, CIDR), port, and protocol. Default deny-all at the bottom.

Priority	Name	Direction	Source	Dest Port	Action
100	Allow-HTTPS	Inbound	Internet	443	Allow
200	Allow-HTTP	Inbound	Internet	80	Allow
300	Block-RDP-from-public	Inbound	Internet	3389	Deny
65500	DenyAllInBound	Inbound	Any	Any	Deny (default)

AZURE LOAD BALANCER

Layer-4 (TCP/UDP) load balancer. The **Standard SKU** supports AZs, outbound rules, and HA ports. The Basic SKU is free but lacks zone redundancy and SLA. Use Application Gateway (Layer 7) when you need URL-based routing, SSL termination, or WAF.

VNET PEERING

Connects two VNets privately over Microsoft backbone — no gateway required, no bandwidth overhead, low latency. Peering is non-transitive: A↔B and B↔C does not mean A↔C. Use Azure Virtual WAN or hub-spoke topology for transitive routing at scale.

> ⚠ **Pitfall**
> Once a VNet is created, you cannot resize its address space without deleting subnets with active resources. Plan your CIDR blocks generously (at least /16 for production VNets) before deployment.

Identity & Access Management (Entra ID / Azure AD)

Microsoft Entra ID (formerly Azure Active Directory) is the cloud-based identity and access management service underpinning all Azure authentication. Every Azure subscription trusts exactly one Entra ID tenant.

Core Identity Concepts

USERS & GROUPS

User objects represent individuals. Groups aggregate users for role assignments. Security groups are used for RBAC; Microsoft 365 groups include mailboxes and Teams. Dynamic membership rules use ODATA filter expressions.

SERVICE PRINCIPALS & MANAGED IDENTITIES

Non-human identities for apps and services. **Managed Identities** (system-assigned or user-assigned) have their credentials managed entirely by Azure — no secrets to rotate, ideal for VM-to-KeyVault or AKS workload identity patterns.

AZURE RBAC

Role assignments bind a *security principal* (user/group/service principal) to a *role definition* (set of permissions) at a *scope* (management group/subscription/RG/resource). Roles are additive — deny assignments override.

BUILT-IN ROLES CHEAT SHEET

Role	Permissions	Common Use
Owner	Full control including RBAC management	Subscription admins
Contributor	Create/manage all resources, no RBAC	DevOps engineers
Reader	View all resources, no modifications	Auditors, read-only access
User Access Admin	Manage RBAC only, no resource access	Delegated RBAC management
Storage Blob Data Contributor	Read/write/delete blobs	App identities accessing Blob
AcrPush / AcrPull	Push or pull container images	CI/CD pipelines, AKS kubelet
Key Vault Secrets User	Read secret values	App identities reading secrets

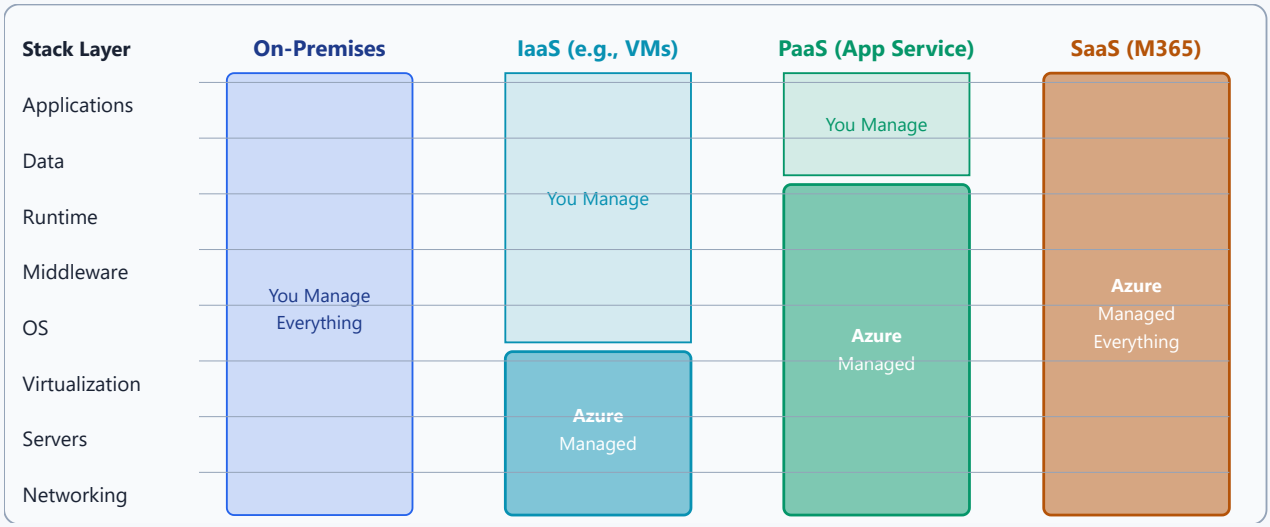
Conditional Access & MFA

Conditional Access policies enforce MFA, device compliance, or block access based on conditions: user group membership, location (named locations / IP ranges), app being accessed, device state (compliant/hybrid-joined), sign-in risk (from Entra ID Protection). This is the primary zero-trust enforcement mechanism.

> **Best Practice**
> Always prefer Managed Identity over service principals with client secrets. When service principals are required, use certificate credentials and rotate them regularly. Store secrets in Azure Key Vault and reference them by URI — never hardcode in environment variables or config files.

IaaS vs PaaS vs SaaS

The service model determines which layers of the stack you manage versus what Microsoft manages. This directly affects TCO, agility, and operational burden.



Shared responsibility model: as you move from IaaS to PaaS to SaaS, Azure takes over progressively more stack layers, reducing your operational burden but also your control.

Model	You Manage	Azure Manages	Azure Examples
IaaS	OS, runtime, middleware, apps, data	Hypervisor, servers, storage, networking	VMs, VMSS, managed disks
PaaS	Applications and data only	OS patches, runtime, scaling, HA infra	App Service, AKS, Azure SQL, Functions
SaaS	User configuration and data	Entire application stack	Microsoft 365, Dynamics 365, Azure DevOps

> **Interview Insight**
> IaaS is preferred when you need OS-level control (custom kernel modules, specific OS tuning), compliance mandates dictating VM-level isolation, or lift-and-shift migrations. PaaS is preferred when developer productivity matters more than infrastructure control, or when rapid scaling and reduced ops are priorities.

Azure Pricing Model & Cost Management

Azure uses a pay-as-you-go model: you pay for what you consume, measured per second/minute/hour depending on the service. Several commitment models reduce cost significantly.

Pricing Dimensions

COMPUTE (VMS)

Billed per second of running time. Size (vCPU + RAM), OS license (Windows costs more), and region affect the rate. Use **Azure Pricing Calculator** for estimates.

STORAGE

Charged on: GB stored per month, number of operations (read/write/list), and data egress (outbound transfer). Ingress to Azure is free.

NETWORKING

VNet creation and inbound bandwidth are free. Outbound data transfer, VNet peering traffic, VPN Gateway, and ExpressRoute circuits are billed separately.

COST OPTIMIZATION STRATEGIES

Strategy	Savings	Commitment	Best For
Reserved Instances (1-yr)	Up to 40%	1 year upfront/monthly	Predictable baseline workloads
Reserved Instances (3-yr)	Up to 65%	3 years upfront/monthly	Stable long-term workloads
Azure Hybrid Benefit	Up to 49%	Bring-your-own Windows/SQL license	On-premises SA license holders
Spot VMs	Up to 90%	Evictable, no SLA	Fault-tolerant batch, ML training
Dev/Test Pricing	Significant	MSDN subscription required	Non-production environments
Auto-shutdown / scale-in	Variable	None	VMs idle overnight/weekends

Azure Cost Management + Billing

Use Azure Cost Management (built into the portal) to set budgets with alert thresholds, analyze spending by resource group/tag/service, and view forecasts. Cost Advisor surfaces actionable recommendations like right-sizing VMs or deleting unused resources.

BASH

```bash
# View cost breakdown for a resource group
az costmanagement query \
  --type Usage \
  --scope "subscriptions/<sub-id>" \
  --dataset-aggregation '{"totalCost":{"name":"PreTaxCost","function":"Sum"}}' \
  --dataset-grouping '[{"type":"Dimension","name":"ResourceGroup"}]' \
  --timeframe MonthToDate
```

⚠ **Pitfall**

Egress costs are frequently overlooked. Transferring data out of Azure (to internet or cross-region) is charged. Architect workloads so compute and data storage are in the same region, and use Azure CDN to cache content close to users rather than serving directly from origin every time.

Azure CLI & Portal

Azure resources can be managed through the Azure Portal (GUI), Azure CLI (cross-platform command-line), Azure PowerShell, REST API, or client SDKs. All methods call the same underlying Azure Resource Manager API.

Azure CLI Essentials

BASH

```bash
# Authenticate and set subscription
az login
az account set --subscription "My Subscription"
az account show

# Resource group operations
az group create --name myRG --location eastus
az group list --output table
az group delete --name myRG --yes --no-wait

# VM operations
az vm create \
  --resource-group myRG \
  --name myVM \
  --image Ubuntu2204 \
  --size Standard_B2s \
  --admin-username azureuser \
  --generate-ssh-keys \
  --zone 1

az vm start --resource-group myRG --name myVM
az vm stop --resource-group myRG --name myVM
az vm deallocate --resource-group myRG --name myVM  # Stops billing for compute

# Query with JMESPath
az vm list --query "[].{Name:name, RG:resourceGroup, Size:hardwareProfile.vmSize}" \
  --output table
```

CLI Output Formats

Format	Flag	Use When
JSON (default)	`-o json`	Parsing in scripts, SDK ingestion
Table	`-o table`	Quick human-readable overview
YAML	`-o yaml`	Config management, GitOps
TSV	`-o tsv`	Feeding values to shell variables
None	`-o none`	Silent execution in automation

Azure Cloud Shell

A browser-based shell available inside the Azure Portal. Comes pre-installed with Azure CLI, Azure PowerShell, kubectl, Terraform, and Helm. State is persisted in an Azure Files share. Useful for quick ad-hoc management without a local tool installation.

> ✦ **Tip**
> Use `az configure --defaults group=myRG location=eastus` to set default resource group and location, saving repetitive flags in subsequent commands.

Infrastructure as Code: ARM Templates & Bicep

Azure Resource Manager (ARM) is the unified deployment and management service. All Azure deployments — portal, CLI, SDK — go through ARM. ARM templates (JSON) and Bicep (domain-specific language) codify infrastructure declaratively.

ARM Template Structure

JSON

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplat
  "contentVersion": "1.0.0.0",
  "parameters": {
    "vmName": {
      "type": "string",
      "defaultValue": "myVM",
      "metadata": { "description": "Name of the virtual machine" }
    }
  },
  "variables": {
    "nicName": "[concat(parameters('vmName'), '-nic')]"
  },
  "resources": [
    {
      "type": "Microsoft.Network/networkInterfaces",
      "apiVersion": "2023-11-01",
      "name": "[variables('nicName')]",
      "location": "[resourceGroup().location]",
      "properties": { ... }
    }
  ],
  "outputs": {
    "vmId": {
      "type": "string",
      "value": "[resourceId('Microsoft.Compute/virtualMachines', parameters('vmName'))
    }
  }
}
```

Bicep: The Modern IaC Language

Bicep is a transparent abstraction over ARM templates — it compiles to ARM JSON but with a cleaner syntax, type safety, modules, and better IDE support. Bicep is the recommended IaC path for new Azure projects (Terraform is popular for multi-cloud).

BASH

```
// Bicep example: Storage account
param location string = resourceGroup().location
param storageAccountName string = 'st${uniqueString(resourceGroup().id)}'

resource storageAccount 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: storageAccountName
  location: location
  sku: {
    name: 'Standard_ZRS'
  }
  kind: 'StorageV2'
  properties: {
    accessTier: 'Hot'
    supportsHttpsTrafficOnly: true
    minimumTlsVersion: 'TLS1_2'
    allowBlobPublicAccess: false
  }
}

output storageAccountId string = storageAccount.id
```

BASH

```
# Compile Bicep to ARM JSON
az bicep build --file main.bicep

# Deploy Bicep directly
az deployment group create \
  --resource-group myRG \
  --template-file main.bicep \
  --parameters storageAccountName=mySA2026

# What-if preview (dry run)
az deployment group what-if \
  --resource-group myRG \
  --template-file main.bicep
```

ARM Deployment Modes: Incremental vs Complete

Best Practice

Use Bicep modules to break large templates into reusable components. Store modules in a shared Bicep registry (Azure Container Registry supports Bicep registries) and version them. Use `what-if` before every production deployment.

Pitfall

ARM/Bicep templates have an 800-deployment history limit per resource group. Old deployments pile up in CI/CD pipelines. Use `az deployment group delete` scripts or the `--name` flag with stable names to overwrite history entries rather than accumulating 800 and hitting the limit.

Azure DevOps

Azure DevOps is an integrated suite of tools for planning, building, testing, and deploying software. It can be used alongside GitHub (Microsoft owns both) or as a standalone DevOps platform.

Azure DevOps Services

AZURE BOARDS

Agile work tracking: Epics, Features, User Stories, Tasks, and Bugs. Kanban boards, backlogs, sprints, and burndown charts. Integrates with GitHub commits and PRs for end-to-end traceability.

AZURE REPOS

Git repositories (unlimited private) or TFVC. Supports pull requests with code review, branch policies (require PR, require CI pass, minimum reviewer count), and protected branches.

AZURE PIPELINES

CI/CD pipeline service. Supports YAML pipelines (pipeline-as-code) and Classic (GUI) pipelines. Can build Linux, Windows, macOS. Integrates with any source: GitHub, Bitbucket, Azure Repos. Multi-stage pipelines with approvals and gates.

AZURE TEST PLANS

Manual and exploratory testing with test case management. Linked to Boards work items. Captures screenshots, video, and system info during test execution for richer bug reports.

AZURE ARTIFACTS

Universal package feeds supporting NuGet, npm, Maven, PyPI, and Universal Packages. Upstream sources allow proxying public registries, caching packages internally, and enforcing provenance.

ENVIRONMENTS & GATES

Deployment environments (dev/staging/prod) with pre/post-deployment approvals. Automatic checks can query Azure Monitor alerts, invoke REST APIs, or run inline scripts before proceeding.

Azure Pipelines YAML Example

YAML

```yaml
trigger:
  branches:
    include:
      - main

pool:
  vmImage: ubuntu-latest

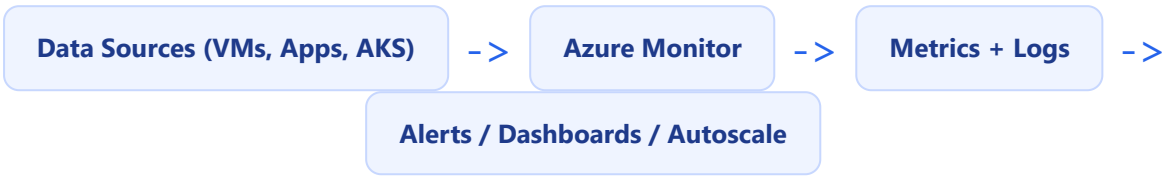
stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - task: UsePythonVersion@0
            inputs:
              versionSpec: '3.11'
          - script: |
              pip install -r requirements.txt
              pytest tests/ --junitxml=junit.xml
            displayName: 'Run tests'
          - task: PublishTestResults@2
            inputs:
              testResultsFiles: 'junit.xml'

  - stage: Deploy
    dependsOn: Build
    condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
    jobs:
      - deployment: DeployWebApp
        environment: production
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebApp@1
                  inputs:
                    azureSubscription: 'MyAzureServiceConnection'
                    appType: webAppLinux
                    appName: 'myAppService'
                    package: '$(Pipeline.Workspace)/**/*.zip'
```

> **Note**
> Azure Pipelines can also trigger GitHub Actions workflows or be triggered by them. Microsoft-hosted agents include 1,800 free minutes/month per organization. Use self-hosted agents for private VNet deployments, custom tooling, or higher throughput.

Monitoring: Azure Monitor & Log Analytics

Azure Monitor is the unified observability platform that collects metrics, logs, and traces from Azure resources, applications, and on-premises infrastructure. It feeds dashboards, alerts, and autoscale rules.



Metrics vs Logs

AZURE MONITOR METRICS

Time-series numerical data (CPU %, memory MB, request count). Collected every 1 minute. Retained for 93 days. Ideal for real-time dashboards, autoscale triggers, and threshold alerts. Query with Metrics Explorer or the REST API.

AZURE MONITOR LOGS (LOG ANALYTICS)

Structured log data stored in a Log Analytics workspace. Queried with Kusto Query Language (KQL). Retained 30 days by default (up to 730 days). Supports custom logs, Syslog, Windows Event logs, and application traces (via App Insights).

Kusto Query Language (KQL) Examples

BASH

```
// Top 5 VMs by average CPU (last hour)
Perf
| where TimeGenerated > ago(1h)
| where ObjectName == "Processor" and CounterName == "% Processor Time"
| summarize AvgCPU = avg(CounterValue) by Computer
| top 5 by AvgCPU desc

// Find failed HTTP requests in App Insights
requests
| where timestamp > ago(24h)
| where success == false
| summarize count() by resultCode, name
| order by count_ desc

// Alert-ready: VMs with disk > 90% used
Perf
| where CounterName == "% Used Space"
| where CounterValue > 90
| summarize arg_max(TimeGenerated, *) by Computer, InstanceName
```

Diagnostic Settings

Each Azure resource can emit Platform Logs (Activity Log, Resource Logs) and Metrics to multiple destinations via Diagnostic Settings: Log Analytics workspace, Storage Account (archival), Event Hub (stream to SIEM), or Azure Monitor partner integrations (Datadog, Splunk). Enable diagnostic settings via CLI, ARM, or Policy.

BASH

```
az monitor diagnostic-settings create \
  --name "myDiagSettings" \
  --resource /subscriptions/<sub-id>/resourceGroups/myRG/providers/Microsoft.Compute/v
  --workspace /subscriptions/<sub-id>/resourceGroups/myRG/providers/Microsoft.Operatio
  --metrics '[{"category":"AllMetrics","enabled":true}]' \
  --logs '[{"categoryGroup":"allLogs","enabled":true}]'
```

Alerts

Azure Monitor Alerts can be triggered by: metric thresholds (static or dynamic/anomaly-based), log search queries (KQL), Activity Log events, Resource Health changes, or Smart Detection (App Insights anomaly detection). Action Groups define the notification/response: email, SMS, webhook, Logic App, Azure Function, ITSM ticketing.

Best Practice

Create a centralized Log Analytics workspace per environment (dev/prod) rather than per resource. Enable the VM Insights and Container Insights solutions for pre-built dashboards. Set up a budget alert in Azure Cost Management alongside performance alerts — cost anomalies are often the first indicator of a misconfigured auto-scale rule.

Security & Compliance Best Practices

Azure security is a shared responsibility. Microsoft secures the cloud infrastructure; you secure what you put in the cloud. Microsoft Defender for Cloud provides unified security posture management across Azure, on-premises, and multi-cloud environments.

Key Security Services

MICROSOFT DEFENDER FOR CLOUD

Provides a Secure Score (0-100%) based on security recommendations. Covers: servers (EDR via Defender for Endpoint), containers (image scanning, AKS threat detection), SQL (brute-force detection, SQL injection alerts), Key Vault, and more. CSPM and CWPP in one pane.

AZURE KEY VAULT

Centralized secrets management for keys, secrets, and certificates. Supports HSM-backed keys (Premium tier). Access is controlled by Entra ID + Key Vault access policies or RBAC. Full audit log via Diagnostic Settings. Integrates natively with App Service, AKS, VMs, and Azure DevOps.

AZURE POLICY

Enforce governance rules at scale. Policies can Audit (report non-compliance), Deny (block deployment), DeployIfNotExists (auto-remediate), or Append (add required tags). Assign built-in or custom policies at management group/subscription/RG scope. Initiatives bundle related policies (e.g., CIS Benchmark initiative).

MICROSOFT SENTINEL

Cloud-native SIEM + SOAR. Ingests signals from Defender products, Azure Activity Log, and third-party connectors. Uses ML-based analytics rules to detect threats. Playbooks (Logic Apps) automate response actions like isolating VMs or blocking user sign-ins.

SECURITY CHECKLIST

Control	Service/Feature	Priority
Enable MFA for all users	Entra ID Conditional Access	Critical
No public IPs on VMs unless required	NSG + Azure Bastion	Critical
Encrypt data at rest	Azure Disk Encryption, Storage SSE	Critical
Store secrets in Key Vault	Azure Key Vault + Managed Identity	Critical
Enable Defender for Cloud (Standard)	Microsoft Defender for Cloud	High
Apply Azure Policy for resource guardrails	Azure Policy + Management Groups	High
Enable resource locks on production RGs	Azure Resource Locks	High
Review Secure Score weekly	Defender for Cloud	Medium

Azure Cheat Sheet: Quick Reference

Service Mapping by Category

Category	IaaS / Low-level	PaaS / Managed	Serverless
Compute	Virtual Machines, VMSS	App Service, AKS	Azure Functions, Container Apps
Storage	Managed Disks, Azure Files	Azure Blob, Azure Data Lake	—
Database	SQL on VM	Azure SQL, Cosmos DB, PostgreSQL Flexible	—
Messaging	—	Service Bus, Event Hubs	Event Grid, Queue Storage
Networking	VNet, NIC, NSG, LB, VPN GW	Application Gateway, Front Door	Azure DNS
Identity	—	Entra ID, B2C	—
AI/ML	NC-series GPU VMs	Azure Machine Learning, AI Foundry	Cognitive Services APIs
DevOps	—	Azure DevOps, GitHub Actions	—
Monitoring	—	Azure Monitor, Log Analytics	Application Insights

Common Azure CLI Patterns

BASH

```
# List all resources in a subscription
az resource list --output table

# Get resource IDs for scripting
az vm list --query "[].id" --output tsv

# Check what's running (and costing money)
az vm list --show-details --query "[?powerState≠'VM deallocated'].{Name:name, State:p

# Lock a resource group
az lock create --name "NoDelete" --lock-type CanNotDelete \
  --resource-group myRG

# List role assignments at subscription scope
az role assignment list --scope /subscriptions/<sub-id> --output table

# Enable a system-assigned managed identity on a VM
az vm identity assign --resource-group myRG --name myVM

# Grant managed identity access to Key Vault
az keyvault set-policy --name myKV \
  --object-id <managed-identity-object-id> \
  --secret-permissions get list
```

Interview Q&A

Common Azure interview questions with concise, technically precise answers for cloud engineering, DevOps, and solution architect roles.

> **What is the difference between an Availability Set and Availability Zones in Azure?**
>
> An Availability Set protects against single-rack (fault domain) and rolling-reboot (update domain) failures within a single physical datacenter. It provides a 99.95% VM SLA. Availability Zones are physically separate datacenters within the same Azure region, each with independent power, cooling, and networking. Distributing VMs across AZs protects against datacenter-level failures and provides a 99.99% VM SLA. AZs are the preferred choice when the region supports them; Availability Sets are a fallback for regions without AZs or for legacy architectures.

> **How does Azure RBAC work, and what is the difference between Owner, Contributor, and Reader roles?**
>
> Azure RBAC (Role-Based Access Control) is an authorization system built on Azure Resource Manager. A role assignment has three components: a security principal (user, group, service

principal, or managed identity), a role definition (a collection of allowed actions/notActions/dataActions), and a scope (management group, subscription, resource group, or resource). Roles are additive — deny assignments explicitly override allows. Owner: full control including the ability to delegate access via RBAC. Contributor: can create and manage all resource types but cannot grant access to others. Reader: can view all resources but cannot make changes.

Explain the difference between NSGs and Azure Firewall. When would you use each?

NSGs are free Layer-4 (TCP/UDP) stateful ACLs applied to subnets or NICs within a VNet. They filter traffic based on IP address, port, and protocol. They lack application-layer awareness, threat intelligence, and centralized management. Azure Firewall is a managed, stateful L3-L7 firewall as a service with built-in high availability and cloud scalability. It adds: FQDN filtering, application rules (URL-based), network rules, threat intelligence-based filtering, TLS inspection, and IDPS (Premium SKU). Use NSGs for micro-segmentation within a VNet (subnet-level controls). Use Azure Firewall in a hub-spoke topology to centrally govern all ingress, egress, and east-west traffic across VNets and on-premises networks.

What is a Managed Identity, and why is it preferred over a service principal with a client secret?

A Managed Identity is an Entra ID identity automatically managed by Azure for a resource (like a VM, App Service, or AKS pod). Azure handles credential rotation — there is no client secret or certificate to store or rotate. The resource's application code calls the Azure IMDS endpoint (169.254.169.254) to get a short-lived token transparently. This eliminates the most common secret sprawl vulnerability (secrets in config files, environment variables, or source code). Service principals with client secrets require manual secret rotation and secure secret storage. System-assigned MIs are tied to a single resource lifecycle; user-assigned MIs can be shared across multiple resources.

What is the purpose of VNet peering, and what are its limitations?

VNet peering connects two VNets over Microsoft's backbone network with low latency and no bandwidth overhead — traffic stays private and never traverses the public internet. Address spaces of peered VNets must not overlap. Peering is non-transitive: if VNet A is peered with B and B is peered with C, traffic cannot flow from A to C without direct peering or routing through a hub (NVA, Azure Firewall, or Virtual WAN). Global VNet peering crosses regions. Peering is free for traffic within the same region, but cross-region peering incurs data transfer charges.

How does Azure Blob Storage Lifecycle Management work, and what tiers are available?

Lifecycle Management policies automatically transition blobs between tiers (Hot → Cool → Cold → Archive) or delete blobs based on rules evaluated once per day. Rules match blobs by prefix or blob index tags and act on: last-modified time, last-access time, or blob creation time. The four tiers are: Hot (frequent access, highest storage cost, lowest access cost), Cool (infrequent access, stored ≥30 days, ~50% cheaper storage than Hot), Cold (rare access, stored ≥90 days), and

Archive (offline, must be rehydrated to Hot/Cool before reading — takes up to 15 hours for Standard priority or 1 hour for High priority). Rehydration from Archive is charged separately and can be expensive if done frequently — plan data access patterns before archiving.

What is the difference between Azure ARM templates and Bicep? When would you choose Terraform instead?

ARM templates are JSON documents that describe Azure infrastructure declaratively. They can be verbose, difficult to read, and error-prone due to JSON's lack of comments and expressions. Bicep is a domain-specific language that compiles to ARM JSON — it offers cleaner syntax, type safety, modules, and IDE support while retaining full ARM capability. Every Bicep feature maps 1:1 to ARM. Choose Bicep when working in Azure-only environments and wanting a Microsoft-supported, first-party IaC tool. Choose Terraform when managing multi-cloud infrastructure (Azure + AWS + GCP), when your team has existing Terraform expertise, or when you need a rich module ecosystem from the Terraform Registry. Both support state management; Terraform's state is external (Blob backend for Azure) while ARM/Bicep state lives in the Azure Resource Manager itself.

What is Azure Monitor Log Analytics, and how do you query it?

Azure Monitor Log Analytics is a managed log aggregation and analytics service backed by a Log Analytics workspace. It stores structured log data from Azure resources, VMs (via agents), AKS (Container Insights), applications (Application Insights), and custom sources. Data is queried using Kusto Query Language (KQL), a read-only, SQL-like language optimized for time-series analysis. Key operators include: `where` (filter), `summarize` (aggregate), `project` (select columns), `join` (combine tables), `extend` (add computed columns), `order by` / `top`. Queries can be used in dashboards (Azure Workbooks), alerts (Log Alert rules), or exported via REST/SDK. Default retention is 30 days; interactive retention can be extended up to 730 days.

Explain the Azure shared responsibility model. What does the customer always own?

In the Azure shared responsibility model, Microsoft is always responsible for physical infrastructure (datacenters, hardware, networking, hypervisor). What the customer manages varies by service model: IaaS — customer manages OS, middleware, runtime, applications, and data. PaaS — customer manages applications and data only (Microsoft handles OS and platform patches). SaaS — customer is responsible only for data, accounts, and access configuration. Regardless of service model, the customer always retains responsibility for: data classification and encryption, identity and access management (who can access what), client endpoint security, and compliance obligations specific to their industry. Microsoft never manages a customer's Entra ID tenant configuration, RBAC assignments, or the data itself.

How do Azure Pipelines stages, environments, and approvals work together for a safe CD process?

In Azure Pipelines YAML, a pipeline is divided into stages (e.g., Build, Deploy-Staging, Deploy-Production). A stage contains jobs, which contain steps. Deployment jobs reference Environments — named logical targets (dev, staging, prod) that track deployment history and can hold approval gates. When a deployment job targets an environment with a required approver configured, the pipeline pauses at that stage and sends notifications to designated approvers. Approvers review deployment details (which commit, which build, test results) and approve or reject. Pre-deployment checks can also call REST APIs, invoke Azure Functions, or query Azure Monitor for active alerts before proceeding. This combination ensures that every production deployment requires explicit human sign-off and passes automated health gates, preventing accidental or unsafe releases.

✓ **Study Tip**

For AZ-900 (Fundamentals), focus on service categories and the shared responsibility model. For AZ-104 (Admin), master VNet configuration, RBAC, and ARM templates. For AZ-204 (Developer), deepen knowledge of App Service, Functions, Cosmos DB, and Key Vault. For AZ-305 (Architect), practice designing multi-region, high-availability, cost-optimized architectures.